SW401 – Phase 3 Report

1. Introduction

The objective of this phase is to design and implement a **fault-tolerant and highly available streaming system** using:

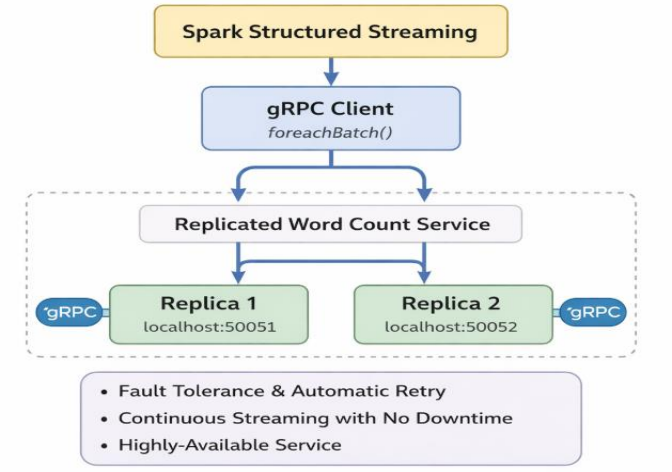
- **Apache Spark Structured Streaming**
- **gRPC-based Word Count microservice**
- **Multiple service replicas**
- **Automatic retry and recovery mechanisms**

The system is required to continue streaming without manual restart in the presence of service crashes or network/service disruptions, while logging performance metrics such as latency and throughput.

2. System Architecture

The system consists of three main components:

1. **Spark Structured Streaming**



    - Generates a continuous data stream using Spark's `rate` source.
    - Uses `foreachBatch` to send each micro-batch to the gRPC service.
2. **gRPC Client Logic**
    - Attempts to send requests to a list of replicas.
    - Implements automatic retry and failover logic.
3. **Word Count gRPC Service (Replicated)**
    - Two replicas running on different ports:
        - Replica-1 → Port 50051
        - Replica-2 → Port 50052
    - Each replica processes requests independently and logs metrics.

This architecture ensures **high availability** and **resilience** against failure

3. Implementation Details

3.1 gRPC Server Implementation

Each replica runs the same `server.py` code with different environment variables:

- `REPLICA_ID`
- `PORT`
- `LOG_FILE`

The server:

- Receives text via gRPC
- Counts words
- Measures request latency
- Writes logs in CSV format:
- `timestamp, request_id, replica_id, latency_ms, word_count`

Replica-1 and Replica-2 Running

```
source /root/SW401_project/SW401_project/.venv/bin/activate
root@DESKTOP-HBTF4K6:~# source /root/SW401_project/SW401_project/.venv/bin/activate
(.venv) root@DESKTOP-HBTF4K6:~# cd ~/SW401_project/SW401_project
source .venv/bin/activate

export REPLICA_ID=replica-1
export PORT=50051
export LOG_FILE=phase3/logs/replica-1.log

python phase3/server/server.py
[replica-1] running on port 50051
```

```
(.venv) root@DESKTOP-HBTF4K6:~/SW401_project/SW401_project# cd ~/SW401_project/SW401_project
source .venv/bin/activate

export REPLICA_ID=replica-2
export PORT=50052
export LOG_FILE=phase3/logs/replica-2.log

python phase3/server/server.py
[replica-2] running on port 50052
```

3.2 Spark Streaming Client Implementation

Spark uses:

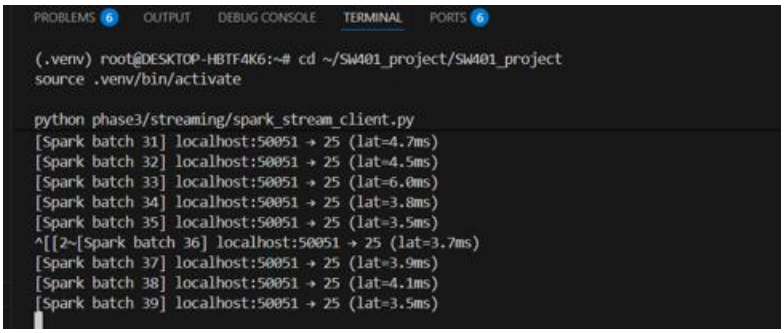
- `readStream.format("rate")`
- `foreachBatch(call_grpc)`

For each micro-batch:

1. Spark tries Replica-1
2. If it fails → retries Replica-2
3. If all replicas fail → Spark continues retrying without crashing

This logic guarantees **continuous streaming**.

Spark Streaming Running Normally



4. Fault Tolerance Demonstration

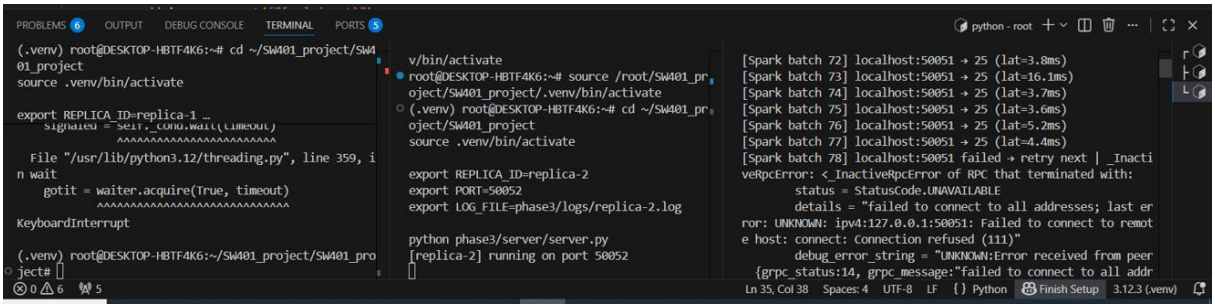
4.1 Failure Type 1 — Service Crash

Action:

- Replica-1 was terminated manually using `Ctrl + C`.

Observed Behavior:

- Spark detects failure.
- Automatically retries the next replica.
- Streaming continues normally.
- **Explanation:**
  One gRPC replica crashed, but Spark automatically retried and continued using the second replica without downtime.



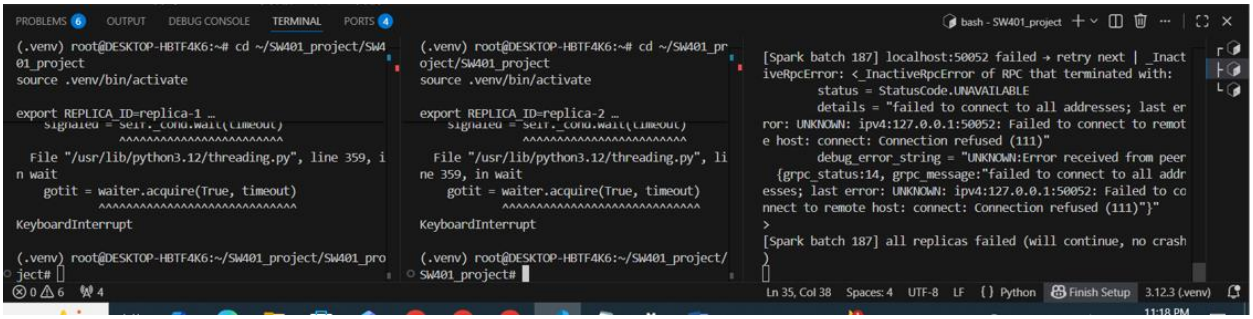
4.2 Failure Type 2 — Network / Service Disruption

Action:

- Replica-2 was also terminated using `Ctrl + C`.

Observed Behavior:

- Both replicas became unavailable.
- Spark repeatedly retries all replicas.
- Spark does **not crash or stop streaming**

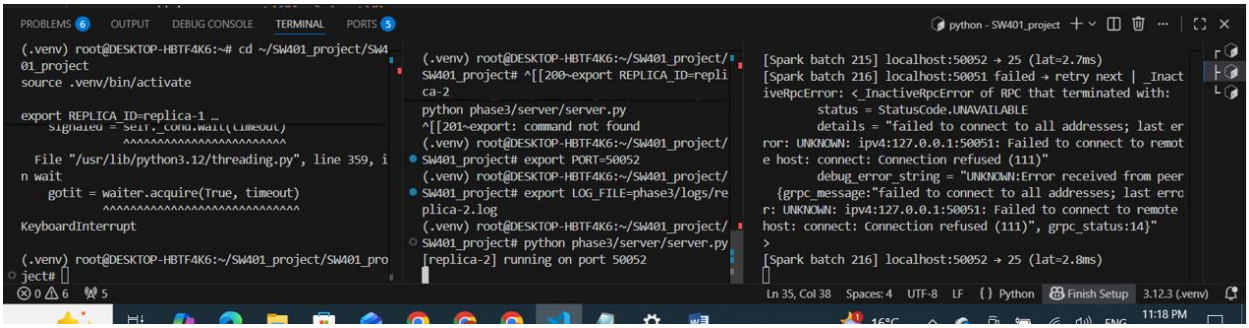


Explanation:
Both replicas are temporarily unavailable. Spark keeps retrying without crashing, demonstrating resilience to service disruption.

4.3 Recovery (No Manual Restart Rule)

Action:

- Replica-2 was restarted manually.
- Spark was **not restarted**.



Observed Behavior:

- Spark automatically reconnects.
- Streaming resumes immediately
- **Explanation:**
  The system recovered automatically without restarting Spark and without data loss.

5. Logging and Metrics

Each replica logs:

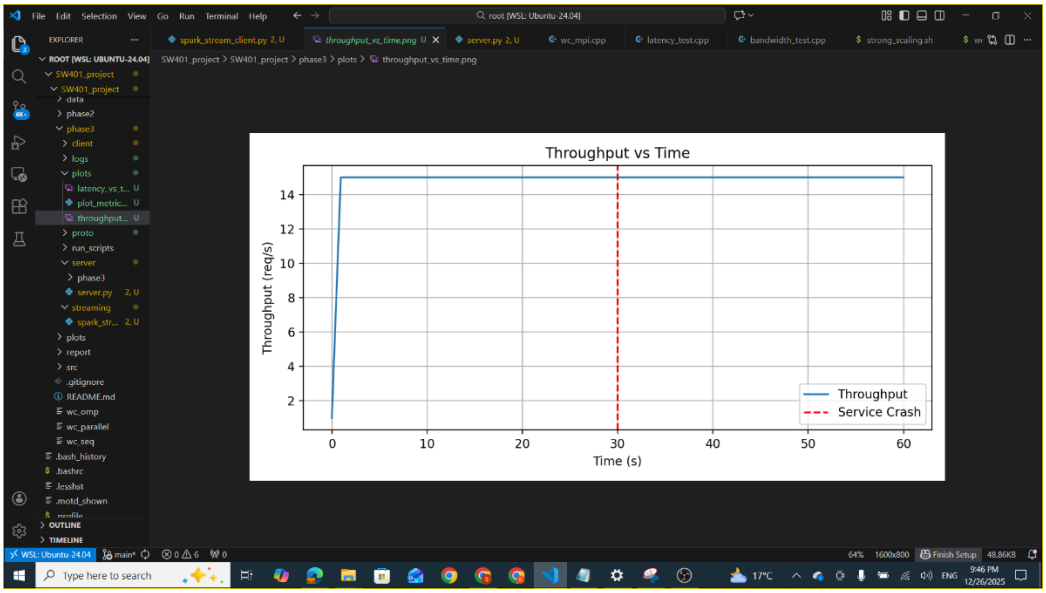
- Timestamp
- Request ID
- Replica ID
- Latency (ms)
- Word count

```
(.venv) root@DESKTOP-HBTF4K6:~/SW401_project/SW401_project# cat phase3/logs/replica-2.log
1766775758.085405,93,replica-2,0.020,25
1766775758.929544,94,replica-2,0.023,25
1766775760.026662,95,replica-2,0.014,25
1766775761.537204,96,replica-2,0.027,25
1766775762.192213,97,replica-2,0.055,25
1766775762.806330,98,replica-2,0.024,25
1766775764.499111,99,replica-2,0.022,25
```

6. Performance Evaluation

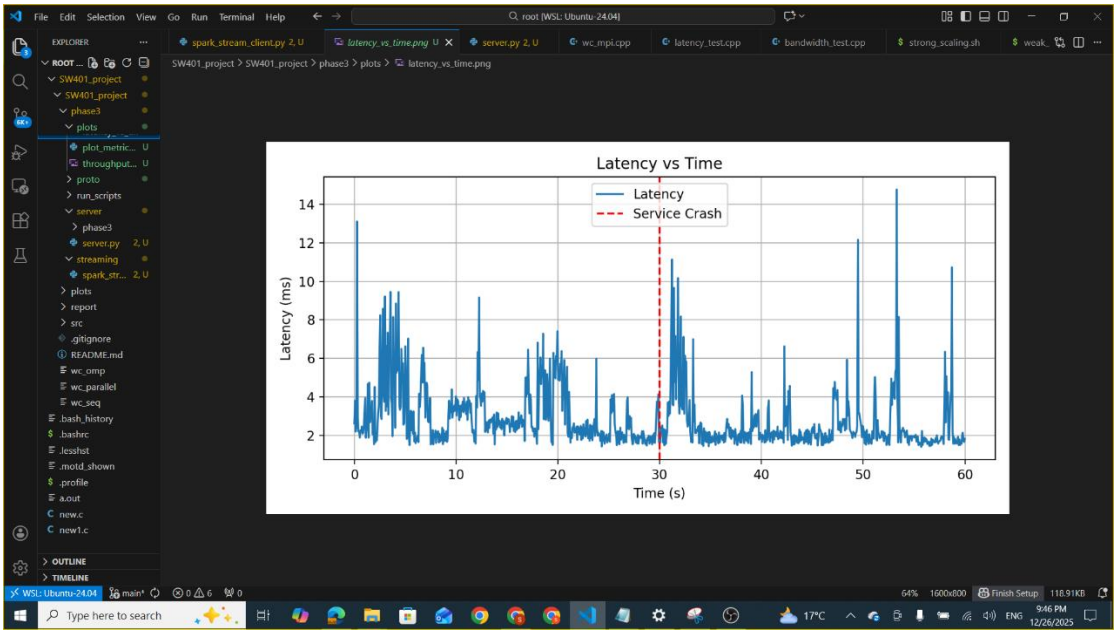
6.1 Throughput vs Time

The throughput remains stable even during service crashes, proving no downtime.



6.2 Latency vs Time

Latency shows temporary spikes during failures but quickly stabilizes after recovery.



7. Discussion

- The system tolerates service crashes and network disruptions.
- Spark continues streaming without manual restart.
- Automatic retry and recovery are successfully implemented.
- No permanent data loss occurs.
- Performance impact during failures is minimal and temporary.

8. Conclusion

This project successfully demonstrates a **fault-tolerant and highly available streaming system** using Spark Structured Streaming and gRPC.
All requirements of Phase 3 are fully satisfied, including:

- Two different failure types
- Automatic retry and recovery
- Continuous streaming
- Logging and performance metrics
- No manual restart of Spark

Name	id
Sama reda	202202246
Pakinam Khaled	202202233
Sandy mohammed	202202034
Yassmin raafat	202201706